Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

Lab Report 4

[Course Code: COMP 307]

Submitted by:
Jatin Madhikarmi
Roll No. 32

Submitted to:
Ms. Rabina Shrestha
Department of Computer Science and Engineering

1 Terminal Output Overview

```
jatin@JatinMadhikarmi:~$ touch notes.txt
jatin@JatinMadhikarmi:~$ ls -l notes.txt
-rw-r--r-- 1 jatin jatin 0 Jan 30 12:07 notes.txt
jatin@JatinMadhikarmi:~$
```

Figure 1: Terminal Output

a. Default Permission String

The full permission string is: **`-rw-r--r--`**

Permission String Breakdown

The string is divided into four distinct parts: the file type indicator and three sets of triplets.

Detailed Breakdown Table

Here is the specific breakdown of each character in the string `-rw-r--r--`:

Position	Character	Category	Permission Type
1	`-`	File Type	Regular File
2, 3, 4	`rw-`	**Owner (User)**	Read, Write, No Execute
5, 6, 7	`r--`	**Group**	Read, No Write, No Execute
8, 9, 10	`r--`	**Others**	Read, No Write, No Execute

Table 1: Character-by-Character Permission Breakdown

Permission Definitions

- **Owner (jatin):** Has `rw-` permissions. This means the user who created the file can view its contents and modify them.
- **Group (jatin):** Has `r--` permissions. Members of the (jatin) group can view the file but cannot edit it.
- **Others:** Has `r--` permissions. Any other user on the system can read the file but has no write access.

Technical Summary

The total permission set can be summarized as follows:

- **Total String:** `-rw-r--r--`
- **Octal Representation:** 644
- **File Owner:** jatin
- **File Group:** jatin

2 Terminal Output Overview

The directory was created and inspected using the following commands:

```
jatin@JatinMadhikarmi:~$ mkdir project
jatin@JatinMadhikarmi:~$ ls -ld project
drwxr-xr-x 2 jatin jatin 4096 Jan 30 12:25 project
jatin@JatinMadhikarmi:~$
```

Figure 2: Terminal Output

Comparison of Permission Strings

There is a significant difference between the file created previously (`notes.txt`) and the new directory (`project`).

Feature	File (notes.txt)	Directory (project)
Full String	`-rw-r--r--`	`drwxr-xr-x`
Type Indicator	`-` (Regular File)	`d` (Directory)
Owner	`rw-` (Read/Write)	`rwx` (Read/Write/Execute)
Group	`r--` (Read Only)	`r-x` (Read/Execute)
Others	`r--` (Read Only)	`r-x` (Read/Execute)

Q) Why Directories Require Execute (x) Permissions?

In Linux, the execute bit (`x`) functions differently for directories than it does for files.

Entering the Directory

For a directory, the **execute** permission is often called the **"search bit."** It is required to *enter* the directory using the `cd` command. Without the `x` bit, you cannot access any files or subdirectories inside, even if you have read permissions.

Accessing Metadata

While the **read (r)** bit allows you to list the names of the files inside (`ls`), the **execute (x)** bit is what allows you to "pass through" the directory to access the actual data or attributes (like file size) of the files within it.

Summary of Directory Permissions

- **Read (r):** Ability to list the contents of the directory.
- **Write (w):** Ability to create, delete, or rename files within the directory.
- **Execute (x):** Ability to enter the directory (`cd`) and access files inside.

3 Command Execution

To restrict access to the file `notes.txt` so that only the owner has full control, the following command is used:

```
jatin@JatinMadhikarmi:~$ ls -l notes.txt
-rw-r--r-- 1 jatin jatin 0 Jan 30 12:07 notes.txt
jatin@JatinMadhikarmi:~$ chmod u+rwx,g-rw,o-rw notes.txt
jatin@JatinMadhikarmi:~$ ls -l notes.txt
-rwx------ 1 jatin jatin 0 Jan 30 12:07 notes.txt
jatin@JatinMadhikarmi:~$
```

Figure 3: Terminal Output

Command Explanation

The `chmod` (change mode) command used here utilizes **symbolic notation** to modify specific permission bits.

Resulting Permission String Breakdown

The new string **`-rwx------`** indicates the following:

Part	Action
`u+rwx`	Adds **Read**, **Write**, and **Execute** permissions for the **User** (owner).
`g-rw`	Removes **Read** and **Write** permissions from the **Group**.
`o-rwx`	Removes **Read**, **Write**, and **Execute** permissions from **Others**.

- **Owner (u):** Has `rwx`. The user *jatin* can read, modify, and run the file as a script/program.
- **Group (g):** Has `---`. Members of the group have **no access** to the file.
- **Others (o):** Has `---`. No other users on the system can see or touch the file.

Technical Note: Octal Equivalent

The symbolic command `u+rwx,g-rw,o-rwx` results in a permission set where the owner has 7 $(4 + 2 + 1)$ and everyone else has 0. This is numerically equivalent to running:

chmod 700 notes.txt

Terminal Output Overview

```
jatin@JatinMadhikarmi:~$ chmod 644 notes.txt
jatin@JatinMadhikarmi:~$ ls -l notes.txt
-rw-r--r-- 1 jatin jatin 0 Jan 30 12:07 notes.txt
jatin@JatinMadhikarmi:~$ chmod 700 notes.txt
jatin@JatinMadhikarmi:~$ ls -l notes.txt
-rwx------ 1 jatin jatin 0 Jan 30 12:07 notes.txt
jatin@JatinMadhikarmi:~$
```

Figure 4: Terminal Output

In the above figure we first revert the notes.txt file to the original permission using the chmod 644 notes.txt (Permission Converstion table is provided down below) and then we use chmod 700 notes.txt to show that this command is equivalent to the above command.

Permission Type Conversion Table

Each permission bit has a specific numeric weight. To find the total for a user category (Owner, Group, or Others), simply add the values of the permissions you wish to grant.

Symbol	Permission	Numeric Value	Function
`r`	Read	4	View file contents / List directory
`w`	Write	2	Modify file / Create or delete files in directory
`x`	Execute	1	Run file as program / Enter (cd) directory
`-`	None	0	No access allowed

Table 2: The Binary Weights of Permissions

Common Permission Combinations

This table shows how the individual weights add up to the single digit used in commands like `chmod 700`.

Numeric Sum	Calculation	Resulting Symbols
7	$4+2+1$	`rwx`
6	$4+2+0$	`rw-`
5	$4+0+1$	`r-x`
4	$4+0+0$	`r--`
0	$0+0+0$	`---`

Example Case: chmod 700

When you run `chmod 700 notes.txt`, you are applying three separate calculations:

1. **First Digit (7):** Owner gets $4+2+1$ (`rwx`).
2. **Second Digit (0):** Group gets $0+0+0$ (`---`).
3. **Third Digit (0):** Others get $0+0+0$ (`---`).

Resulting String: `-rwx------`

4 Experimental Case: Group-Only Access

To test restricting access to only the group, the following octal command was used on the directory

```
jatin@JatinMadhikarmi:~$ chmod 050 project
jatin@JatinMadhikarmi:~$ ls -ld project
d---r-x--- 2 jatin jatin 4096 Jan 30 12:25 project
jatin@JatinMadhikarmi:~$
```

Figure 5: Terminal Output

Analysis of Result

The resulting permission string `d---r-x---` confirms that:

- **Owner (0):** The owner has been locked out of their own directory.
- **Group (5):** Group members can `ls` (read) the directory and `cd` (execute) into it.
- **Others (0):** Outside users have no access.

Digit	Value	Calculated Binary
1st (User)	0	`---`
2nd (Group)	5	`r-x` $(4+1)$
3rd (Others)	0	`---`

Table 3: Logic for chmod 050

5 The Risks of `chmod 777`

The command `chmod 777` grants **read (4), write (2), and execute (1)** permissions to three distinct tiers of users: the Owner, the Group, and *Others* (everyone else on the system). It is generally discouraged for the following reasons:

- **Security Vulnerability:** Anyone with access to the system can modify or delete the file. In a web server environment, if a directory is set to 777, an attacker who gains access to a low-privileged service can upload malicious scripts and execute them.
- **Data Integrity:** It removes the ”safety net” that prevents accidental modification or deletion of critical system files by non-administrative processes.
- **Principle of Least Privilege (PoLP):** This security pillar dictates that a user or process should only have the minimum permissions necessary to perform its task. `777` is the literal opposite of this principle.

6 When to Use `chgrp` instead of `chown`

While `chown` (change owner) is a powerful tool that can also change groups, `chgrp` is more appropriate in specific collaborative scenarios:

6.1 Collaborative Workflows

When multiple users need to work on the same set of files (e.g., a shared project directory), you often want the **original creator** to remain the owner for accountability or quota reasons. By using `chgrp`, you can grant access to a specific team (the group) without stripping the original author of their ownership status.

6.2 Delegated Administration

In many systems, a non-root user who owns a file can use `chgrp` to change the file's group to any group they are a member of. However, most modern Linux systems prevent non-root users from using `chown` to give a file away to another user (to prevent quota dodging or malicious hand-offs). Thus, `chgrp` is the tool of choice for regular users managing shared access.

6.3 Standard Security Hardening

It is common practice to keep a service account as the `owner` (with full permissions) but change the `group` to a restricted group that only has read-only access for logging or monitoring tools.